

Veille métier et technologique

Pierre-Antoine Morin
OpenClassrooms – Data Analyst – Projet 13 – Mission 1

Table des matières

Introduction.....	2
Méthodologie	2
Conception du prompt	2
Comparaison des deux agents IA	2
Sélection d'outils proposés par l'IA	3
Qualité du nettoyage.....	3
Accélérer l'analyse exploratoire des données.....	3
Renforcer l'interprétabilité.....	4
Fiabiliser le déploiement et la reproductibilité	4

Introduction

La première mission de ce projet consiste à reprendre le notebook Jupyter développé dans le cadre d'un autre projet et de l'améliorer. Pour ce faire, il est recommandé d'utiliser l'intelligence artificielle, à la fois comme source de propositions dans le cadre d'une veille métier et technologique, mais aussi comme assistant de code.

Ce document indique comment j'ai utilisé l'IA pour la partie veille, en insistant sur la méthodologie suivie, la traçabilité et l'interprétation critique des résultats.

Méthodologie

Tout d'abord, j'ai commencé par me replonger dans l'ancien projet afin de me réapproprier le contexte, les enjeux, les solutions implémentées ainsi que les recommandations finales.

À partir de cela, j'ai sélectionné les pistes d'amélioration qui me semblent pertinentes dans la perspective d'améliorer un notebook Python existant, en prenant en compte les données disponibles et les contraintes sur l'environnement de développement.

Cette analyse m'a permis de concevoir un prompt que j'ai envoyé non pas à un seul mais à deux agents IA, afin de m'inscrire dans une démarche comparative permettant de maximiser le nombre d'opportunités.

Conception du prompt

Afin d'obtenir les meilleures réponses possibles, la qualité de la question envoyée à l'agent IA est déterminante. Pour cela, j'ai donné un maximum d'informations sur le contexte, sans révéler de données sensibles (e.g. valeurs trop précises, nom du client), tout en indiquant clairement ce dont j'ai besoin (veille sur les meilleures pratiques et les outils actuels), dans quel écosystème (intégration Python), avec des critères de comparaison variés et quelques pistes d'axe d'amélioration.

Le prompt final est disponible dans un fichier séparé (format Markdown).

Comparaison des deux agents IA

J'ai envoyé le prompt à deux agents IA : ChatGPT et Vibe (Mistral). Leurs réponses sont disponibles des fichiers séparés (format Markdown). Voici un tableau récapitulatif permettant de les comparer selon divers critères.

Critère	ChatGPT	Vibe (Mistral)	Vainqueur
Quantité d'outils proposés	Très élevée	Élevée	ChatGPT
Syntaxe de code ?	Oui	Non	ChatGPT
Pertinence	Très élevée	Élevée	ChatGPT
Prise d'initiative	Très élevée	Moyenne	ChatGPT
Réputation	Très connu	Moins connu	ChatGPT
Clarté	Très clair	Très clair	-
Liens vers sources fiables ?	Non	Oui	Vibe (Mistral)
Modèle Open Source ?	Non	Oui (partiel)	Vibe (Mistral)
Entreprise (localisation)	OpenAI (USA)	Mistral AI (France)	Vibe (Mistral)

Les deux agents ont proposé des outils pertinents, avec des exemples d'utilisation personnalisée, regroupés par catégorie, avec une hiérarchisation selon les critères demandés et des recommandations avec un ordre de priorité, de manière claire et structurée, dans un style concis et professionnel.

ChatGPT a proposé plus d'outils, notamment récents, avec des exemples de syntaxe de code, et a particulièrement bien compris le besoin exprimé, en étant force de proposition sur les axes d'amélioration, mais sans citer de sources.

Vibe (Mistral) a proposé moins d'outils et s'est limité aux axes d'amélioration donnés dans le prompt. Cependant, il a fourni des liens vers des sources fiables (documentation officielle, GitHub) en utilisant un ton plus pédagogique.

Sélection d'outils proposés par l'IA

Voici une liste de 8 outils proposés par les agents IA qui semblent les plus pertinents pour ce projet, regroupés 2 par 2 selon 4 possibilités d'axe d'amélioration. Seule une présentation générale est proposée dans ce document : pour des exemples d'utilisation personnalisée et de syntaxe de code, se référer aux réponses des agents IA (fichiers Markdown).

Qualité du nettoyage

Pandera permet de décrire un schéma de DataFrame et d'y associer des contraintes directement en Python. Il est particulièrement adapté à la situation, car les règles intra-colonne et inter-colonnes ont déjà été identifiées. Il est assez rapide à mettre en place à partir des données déjà disponibles et très utile pour identifier le plus tôt possible des données incohérentes lors d'une extraction de données sur une autre période.

Great Expectations est davantage orienté vers un dispositif complet de validation et de documentation : jeux d'attentes, validations, checkpoints et rapports lisibles. Ses « Data Docs » peuvent servir de preuve de qualité pour les métiers et de documentation historique des contrôles. La documentation décrit notamment les Expectations comme des assertions vérifiables et les Data Docs comme des rapports HTML regroupant attentes et résultats de validation.

Outil retenu pour la qualité du nettoyage : **Pandera**

- Plus léger, adapté pour une intégration dans un notebook : prioritaire pour une première mise en place
- Implémentation complète des règles métiers possible directement au niveau des DataFrames

Accélérer l'analyse exploratoire des données

Polars est une bibliothèque Python pour la manipulation de données, optimisée pour la vitesse (écrite en Rust). C'est une alternative à Pandas, avec une API similaire mais plus performante pour les gros datasets (notamment en termes de consommation de mémoire) mais toutefois moins bien intégrée avec les librairies statistiques ou de visualisation. Elle propose aussi d'autres fonctionnalités comme l'évaluation paresseuse (exécution optimisée d'une chaîne de traitement complexe à la demande) ou un typage de colonne strict.

DuckDB est un moteur de bases de données embarqué (sans serveur, comme SQLite) orienté OLAP (OnLine Analytical Processing, en opposition à OLTP OnLine Transactional Processing) adapté pour des analyses SQL haute performance en local. Intégration possible en Python avec Pandas et Polars.

Outil retenu pour accélérer l'analyse exploratoire des données : **Polars**

- Polars se prête mieux à une intégration dans un notebook (langage Python).
- DuckDB serait plus adapté pour des analyses en SQL ou une connexion directe aux bases de données pour éviter d'avoir à réaliser une extraction de fichiers.

Renforcer l'interprétabilité

Pingouin est une alternative à d'autres librairies statistiques (e.g. SciPy ou StatsModels) adaptée à des cas d'usages standards dans le cadre d'une analyse descriptive et inférentielle (e.g. validation d'hypothèses, corrélation) sans devoir écrire beaucoup de code statistique bas niveau grâce à une API plus intuitive.

SHAP (SHapley Additive exPlanations) est une bibliothèque Python pour expliquer des relations dans les données et les prédictions des modèles de Machine Learning (régression, classification) via la théorie des jeux coopératifs. Cela permet par exemple de quantifier la contribution des variables d'entrée par rapport à la variable prédite. Cela suppose qu'un modèle de Machine Learning a déjà été mis en place (e.g. avec Scikit-learn).

Outil retenu pour renforcer l'interprétabilité : **Pingouin**

- Pingouin est idéal pour implémenter des tests statistiques permettant de consolider l'interprétation des observations issues de l'analyse des stocks et des ventes.
- SHAP se combine bien avec Scikit-learn pour interpréter des modèles de Machine Learning pour faire de l'analyse prédictive. Or, le faible volume de données disponibles et l'absence de dates de transactions limitent les possibilités de mise en place de tels modèles pour le moment.

Fiabiliser le déploiement et la reproductibilité

Prefect est un outil d'orchestration simple directement intégré dans Python. Il permet d'automatiser des pipelines de nettoyage et de planifier leur exécution à la fréquence désirée. Le workflow ainsi défini est lui-même versionné.

dbt (data build tools) permet de définir des data pipelines (ETL = extract/transform/load) via des modèles SQL (source, staging, intermediate, mart) de manière reproductible et versionnée. Les modèles peuvent être testés et documentés afin de garantir la qualité de la donnée de bout en bout.

Outil retenu pour fiabiliser le déploiement et la reproductibilité : **Prefect**

- Prefect permet par exemple d'automatiser l'extraction mensuelle des fichiers d'entrée (attention, données des autres mois non disponibles ; préférer un script Python plutôt qu'un notebook Jupyter)
- dbt est plus adapté dans un environnement SQL avec un accès direct aux données.